Universidad Rafael Landívar

Campus San Alberto Hurtado, S.J. de Quetzaltenango

Curso: Programación Web · Ing. Manuel Rojas

Actividad 23

Eventia: de una API insegura a un diseño seguro

Entregables completos de la auditoría, rediseño, permisos, OpenAPI, controles y respuestas.

Enlace al repositorio actualizado

https://github.com/carloshugoeg/Eventia

El contenido de los entregables continúa a continuación.

1. Auditoría de las siete APIs

Se analizaron las siete APIs del enunciado. Algunas presentan más de un riesgo; se indica el principal según OWASP API Security Top 10.

API	Problema identificado	Riesgo OWASP API	Solución propuesta
API 1 GET /api/inscripciones/125	Solo comprueba que exista la inscripción; no verifica pertenencia al solicitante (IDOR).	API1 Broken Object Level Authorization	Exigir autenticación y que inscripcion.usuarioId == usuario.id. Responder 403/404 sin filtrar datos ajenos.
API 2 POST /api/usuarios	Mass assignment: el cliente envía rol administrador y el backend lo persiste.	API3 Broken Object Property Level Authorization (también API5)	Lista blanca de campos; rol por defecto; rechazar administrador desde el cliente.
API 3 POST /api/eventos	Cupo/precio negativos sin validar; cualquier autenticado puede crear eventos.	API4 Unrestricted Resource Consumption + API5 Broken Function Level Authorization	Validar cupo > 0 y precio >= 0; exigir rol organizador; rechazar propiedades no permitidas.
API 4 PUT /api/eventos/25	Verifica rol organizador pero no ownership del evento.	API1 Broken Object Level Authorization	Comprobar evento.organizadorId == usuario.id además del rol.
API 5 POST .../inscripciones	Sin tope de cantidad por usuario; cupos inseguros bajo concurrencia.	API4 Unrestricted Resource Consumption / diseño inseguro	Limitar cantidad (p. ej. 1); reserva atómica/condicional en transacción.
API 6 GET /api/usuarios/35	Devuelve passwordHash y campos sensibles aunque el front solo usa nombre/correo.	API3 Broken Object Property Level Authorization	DTO público sin secretos; datos sensibles solo para titular o admin.
API 7 POST /api/auth/login	Intentos ilimitados; mensajes distintos revelan si el correo existe.	API2 Broken Authentication (+ API8 por mensajes)	Rate limiting; mensaje genérico; hashing con tiempo comparable.

Consecuencias: fuga de PII (API 1/6), elevación a administrador (API 2), inventario corrupto y spam (API 3), sabotaje de eventos ajenos (API 4), acaparamiento/sobrecupo (API 5), fuerza bruta y enumeración de cuentas (API 7).

2. Rediseño de las siete operaciones

Las rutas del MVP Eventia usan inglés (/api/events, /api/registrations, …). El rediseño conserva la intención de cada API del enunciado.

API 1 — Consulta de inscripción

- Método/ruta: GET /api/registrations (listado propio).
- Función: devolver solo inscripciones del usuario autenticado.
- Quién: usuario registrado (titular).
- Validaciones: sesión válida; rechazar userId ajeno.
- Autorización: recursos con user_id === sesión.id.
- Respuesta: 200 con lista propia. Errores: 401, 403, 404.

API 2 — Registro de usuarios

- Método/ruta: POST /api/auth/register.
- Datos: name, email, password; role opcional ∈ {participant, organizer}.
- Validaciones: correo válido; contraseña 8–128; lista blanca de rol; nunca administrador.
- Respuesta: 201 + cookie. Errores: 400, 409, 429.

API 3 — Creación de eventos

- Método/ruta: POST /api/events.
- Acceso: organizador autenticado.
- Datos: title, description, category, location, starts_at, capacity.
- Controles: autenticación, rol, cupo 1–10000, fecha futura, categoría en lista.
- Respuestas: 201 / 400 / 401 / 403.

API 4 — Modificación de eventos

- Método/ruta: PATCH /api/events/{id}.
- Acceso: organizador dueño (rol + ownership).
- Controles: organizer_id === user.id; cupo >= ocupados.
- Respuestas: 200 / 400 / 401 / 403 / 404.

API 5 — Inscripciones

- Método/ruta: POST /api/events/{id}/registrations.
- Datos: quantity opcional (solo 1).
- Controles: quantity >= 1 y = 1; transacción; UPDATE condicional de cupo.
- Respuestas: 201 / 400 / 401 / 409.

API 6 — Consulta de usuarios

- Método/ruta: GET /api/auth/me.
- Respuesta pública: { id, name, email, role } o null. Nunca password_hash.

API 7 — Inicio de sesión

- Método/ruta: POST /api/auth/login.
- Controles: rate limit 30/IP/min; mensaje genérico; scrypt + timingSafeEqual.
- Respuestas: 200 / 400 / 401 genérico / 429.

3. Matriz de roles y permisos

Tener un rol no equivale a estar autorizado sobre un recurso concreto. El organizador solo modifica eventos propios.

Operación	Visitante	Usuario	Organizador	Administrador*
Consultar eventos públicos	Permitido	Permitido	Permitido	Permitido
Crear eventos	Denegado	Denegado	Permitido	Permitido
Modificar eventos propios	Denegado	Denegado	Permitido si es dueño	Permitido
Modificar eventos ajenos	Denegado	Denegado	Denegado	Permitido
Inscribirse en eventos	Denegado	Permitido	Permitido	Permitido
Consultar inscripciones propias	Denegado	Solo las suyas	Solo las suyas	Permitido
Consultar inscritos de un evento (org.)	Denegado	Denegado	Si es dueño del evento	Permitido
Administrar usuarios	Denegado	Denegado	Denegado	Permitido

* En el MVP actual no existe rol administrador en runtime; la columna define la política objetivo.

4. Documentación OpenAPI (tres endpoints)

Archivo en el repositorio: docs/openapi-actividad-23.yaml

Coincide con permisos y validaciones de la Fase 2 (rediseño).

Endpoint	Acceso	Autenticación	Códigos clave
POST /api/events	Organizador	Cookie eventia_session	201, 400, 401, 403
PATCH /api/events/{eventId}	Organizador dueño	Cookie eventia_session	200, 400, 401, 403, 404
POST /api/events/{id}/registrations	Usuario autenticado	Cookie eventia_session	201, 400, 401, 409

```
# Fragmento OpenAPI 3.0.3
paths:
  /api/events:
    post:
      security: [{ sessionCookie: [] }]
      summary: Publicar un evento
      responses:
        '201': { description: Evento creado }
        '403': { description: Sin rol organizador }
  /api/events/{eventId}:
    patch:
      security: [{ sessionCookie: [] }]
      summary: Modificar evento propio (ownership)
      responses:
        '200': { description: Actualizado }
        '403': { description: Evento ajeno }
  /api/events/{eventId}/registrations:
    post:
      security: [{ sessionCookie: [] }]
      summary: Inscribirse (quantity = 1)
      responses:
        '201': { description: Confirmada }
        '400': { description: INVALID_QUANTITY }
```

5. Evidencias de dos controles implementados

Control A — Ownership en modificación de eventos (API 4)

Elemento	Detalle
Comportamiento anterior	Riesgo de que un organizador altere eventos ajenos si solo se valida el rol.
Problema de seguridad	OWASP API1 Broken Object Level Authorization.
Modificación realizada	PATCH /api/events/:id exige event.organizer_id === user.id en el servicio.
Prueba con datos válidos	Dueño actualiza título → HTTP 200.
Prueba no autorizada	Otro organizador o participante → HTTP 403 FORBIDDEN.
Resultado	Tests: fase 3 ownership + HTTP PATCH ajeno. Archivos: events/service.js, http/server.js.

Control B — Cantidad de inscripción y anti-sobrecupo (API 5)

Elemento	Detalle
Comportamiento anterior	Riesgo de quantity arbitraria y carrera de cupos.
Problema de seguridad	OWASP API4 Unrestricted Resource Consumption / diseño inseguro.
Modificación realizada	quantity entero >= 1 y solo 1; reserva SQL condicional en transacción.
Prueba con datos válidos	quantity: 1 u omitido → HTTP 201.
Prueba inválida	0, -1 o 2 → 400 INVALID_QUANTITY; cupo agotado → 409; 8 hilos vs 3 cupos → solo 3 OK.
Resultado	Tests fase 3 quantity + concurrencia. Archivos: registrations/service.js, events/service.js.

Evidencia automatizada (npm test): 19/19 pruebas pasaron, incluidas las de Fase 3 / Actividad 23.

```
# Subtest: fase 3: solo el organizador dueño puede modificar su evento
ok 14 - fase 3: solo el organizador dueño puede modificar su evento
# Subtest: fase 3: la cantidad de inscripción debe ser mayor que cero y no se permite sobrecupo
ok 15 - fase 3: la cantidad de inscripción debe ser mayor que cero y no se permite sobrecupo
# Subtest: fase 3: no se pueden consultar inscripciones privadas de otro usuario
ok 16 - fase 3: no se pueden consultar inscripciones privadas de otro usuario
# Subtest: HTTP fase 3: PATCH ajeno, cantidad inválida y userId ajeno se rechazan
ok 17 - HTTP fase 3: PATCH ajeno, cantidad inválida y userId ajeno se rechazan
# Subtest: 8 conexiones concurrentes compiten por 3 cupos: solo 3 inscripciones y 3 confirmaciones
ok 19 - 8 conexiones concurrentes compiten por 3 cupos: solo 3 inscripciones y 3 confirmaciones
# tests 19
# pass 19
# fail 0
```

6. Enlace al repositorio

https://github.com/carloshugoeg/Eventia

Documentación adicional en el repo: docs/actividad-23-seguridad-apis.md, docs/openapi-actividad-23.yaml, docs/fase-3-controles-seguridad.md.

7. Respuestas a las preguntas finales

1. ¿Cuál de las siete APIs representa el mayor riesgo?

La API 2 (registro con asignación de rol). Un atacante se convierte en administrador y puede abusar del resto de operaciones. Las APIs 1 y 6 son graves por fuga de PII/hashes, pero la elevación de privilegios compromete el control de toda la plataforma.

2. ¿Por qué utilizar JWT no garantiza que una API esté protegida?

JWT solo acredita identidad/firma (autenticación). Sin autorización por recurso, validación, rate limiting y diseño de permisos, un token válido puede leer o modificar objetos ajenos.

3. ¿Qué diferencia existe entre validar los datos y autorizar una operación?

Validar comprueba forma y reglas del payload (p. ej. cupo > 0). Autorizar decide si ese sujeto puede ejecutar la acción sobre ese recurso (p. ej. si es el dueño del evento).

4. ¿Por qué documentar la autenticación en OpenAPI no significa que la seguridad ya esté implementada?

OpenAPI es contrato/documentación. Los controles reales viven en el código (middleware, servicios, SQL). Un esquema con security no rechaza por sí solo una petición.

5. ¿Qué controles mantener al evolucionar a servicios independientes?

Autenticación centralizada o tokens cortos; autorización en cada servicio; validación en cada borde; rate limiting; auditoría; secretos fuera del código; TLS; least privilege entre servicios (identidades de servicio / mTLS).

Repositorio: https://github.com/carloshugoeg/Eventia

Generado para la Actividad 23 · Programación Web · Eventia